

Stage 2 Annotation Role Contract 架构设计

1. 文档状态

本文档用于定义 NL2SPL 编译管线中 Stage 2 Annotation 机制的最新架构边界。

本文档的核心结论是：

Annotation 机制不需要推翻。
Role Contract 机制也不是全新设计。
当前代码中已经存在 Role Contract 的雏形。

真正需要做的是：
统一现有 Role Contract source of truth；
强化 validator 的全字段一致性校验；
让所有 annotation 生成路径都经过同一套 role contract；
禁止 downstream 通过 construct_target / slot_target 单字段推断 construct demand；
将 requiredness 三值从 role contract 中剥离，单独作为结构化来源问题处理。

本设计不直接定义具体实现步骤、文件改动或测试计划。它只定义问题背景、职责边界、架构原则和设计决策。

2. 背景问题

Stage 2 Field Router 的职责是将 source span 转换成后续编译阶段可以消费的结构化 `RouteAnnotation`。

当前 Annotation 中包含多个字段：

```
semantic_role
field
route_family
construct_target
slot_target
executable
source_section_id
source_packet_id
reason
metadata
```

这些字段并不属于同一性质。

其中：

semantic_role

是语义判断字段。它回答：

这个 span 在源文本中扮演什么语义角色？

例如：

```
profile_domain
input_contract
output_contract
process_step
failure_mode
exception_handler_action
worker_handoff_candidate
handoff_condition
api_candidate
```

而以下字段更接近编译契约字段：

```
field
route_family
construct_target
slot_target
executable
```

它们回答：

```
如果 semantic_role 已经确定，
这个 span 应进入哪个 compiler field？
属于哪个 route family？
是否对应某个 construct？
对应哪个 construct slot？
是否 executable？
```

这些字段之间存在强约束关系。如果让 LLM 同时自由输出这些字段，就会产生 schema-valid 但 semantic-invalid 的 annotation。

典型问题是：

```
semantic_role=profile_domain
construct_target=RESOURCE_CONTRACT
slot_target=input
```

这在枚举值层面可能都是合法值，但在语义层面不可成立。`profile_domain` 表示任务领域或背景描述，不应同时成为 resource contract input。

3. 核心问题判断

当前系统的问题不是“依赖 Annotation 机制”本身，而是 Annotation 的生成和消费边界不够严密。

具体表现为五类问题：

3.1 LLM 决定的字段过多

LLM 同时决定：

```
semantic_role
field
route_family
construct_target
slot_target
executable
```

这会扩大组合空间。只要其中一个字段受 prompt 词面干扰，annotation 就可能内部矛盾。

3.2 Role Contract 已存在但分散

当前系统中已经存在 role contract 的机制雏形：

```
一份 contract 用于 role -> compiler fields 派生 / normalization;
另一份 contract 用于 validator 检查。
```

问题是这两份 contract 并不同步，覆盖范围也不同。

因此，本设计不主张“新增第三份 Role Contract 表”，而是主张：

```
将已有 role contract 收敛为唯一 source of truth。
```

3.3 Validator 覆盖不完整

当前 validator 的 role contract 检查应覆盖：

```
field
route_family
construct_target
slot_target
executable
```

但如果 validator 只检查部分字段，例如只检查 `executable=False`，则无法发现：

```
profile_domain + RESOURCE_CONTRACT/input
```

这类矛盾。

此外，validator 必须能表达：

```
construct_target 必须为 None  
slot_target 必须为 None
```

`None` 不是“不检查”，而是一种明确 contract expectation。

3.4 Deterministic annotation path 与 LLM refinement path 不统一

系统中存在不同来源的 annotation：

```
LLM refinement annotation  
deterministic packet annotation  
adapter-derived annotation
```

如果只有 LLM path 经过 role contract normalization，而 deterministic path 走另一套映射表，那么系统仍然会出现多套语义来源。

最终架构必须保证：

```
所有 annotation 生成路径都使用同一套 Role Contract。
```

3.5 Downstream 存在单字段 construct 推断风险

任何下游阶段如果使用：

```
construct_target == RESOURCE_CONTRACT
```

或：

```
route_family == resource_contract
```

作为 resource demand existence 的独立依据，都会被矛盾 annotation 击穿。

resource contract demand 的主依据必须是：

```
semantic_role=input_contract
semantic_role=output_contract
```

`construct_target`、`slot_target`、`route_family` 只能作为一致性校验字段，不能单独成为 demand 入口。

4. 设计目标

本设计目标如下：

1. 保留 Annotation 作为 Stage 2 到下游阶段的核心结构化语义载体。
2. 将 LLM 输出收敛到最小语义判断，尤其是 `semantic_role`。
3. 将 `field`、`route_family`、`construct_target`、`slot_target`、`executable` 交给确定性 Role Contract 派生。
4. 统一当前分散的 role contract 机制，形成单一 source of truth。
5. 让 validator 使用同一份 role contract 进行全字段一致性检查。
6. 让 deterministic annotation path 与 LLM refinement path 共享同一套 contract。
7. 明确 `construct_target` 和 `slot_target` 不是独立 truth source。
8. 防止 downstream stage 通过单字段推断 construct demand。
9. 将矛盾 annotation 转换为 visible diagnostic，而不是静默消费或静默丢弃。
10. 将 requiredness 三值作为独立问题处理，不混入 semantic_role 的派生逻辑。

5. 非目标

本设计不承担以下职责：

1. 不推翻 Annotation 机制。
2. 不重新设计 ResourceContractDemandView。
3. 不重新设计 IRS。
4. 不定义具体实现步骤。
5. 不定义具体测试计划。
6. 不要求一次性重写 Stage 2。
7. 不要求立即改动所有 downstream consumers。
8. 不让 DemandView 读取 raw NL。
9. 不让 DemandView 读取 section title 或 evidence text 做语义恢复。
10. 不让 Stage 6 重新判断 requiredness。
11. 不通过 prompt 强化替代 deterministic validation。
12. 不将 requiredness 三值问题假装已经解决。

6. 最新架构结论

6.1 Annotation 仍是核心语义载体

Stage 2 仍然负责产生 confirmed `RouteAnnotation`。

下游组件不应直接消费 raw LLM output，而应消费经过 role contract normalization 和 validation 的 confirmed annotation。

可信边界如下：

```
Raw LLM output
-> 低可信，只是候选语义

Normalized / validated RouteAnnotation
-> 中高可信，可供 Stage 3、DemandView、ConstructPlan 等消费

DemandView / ConstructPlan
-> construct-level source demand，属于更高层投影视图
```

6.2 LLM 只负责 semantic_role

LLM 最适合判断：

```
这个 span 的语义角色是什么？
```

因此，LLM 应主要输出：

```
span_id
semantic_role
reason
primary / ambiguity hint
split hint if needed
```

LLM 不应拥有以下字段的最终决定权：

```
field
route_family
construct_target
slot_target
executable
```

如果为了兼容旧 schema，LLM 仍然输出这些字段，它们也只能作为 debug hint 或候选字段，不能作为 source of truth。

6.3 Role Contract 派生 compiler fields

Role Contract 是确定性契约：

```
semantic_role -> compiler-facing annotation fields
```

例如：

```
semantic_role=profile_domain
-> field=domain
-> route_family=profile
-> construct_target=None
-> slot_target=None
-> executable=False

semantic_role=input_contract
-> field=resources
-> route_family=resource_contract
-> construct_target=RESOURCE_CONTRACT
-> slot_target=input
-> executable=False

semantic_role=output_contract
-> field=resources
-> route_family=resource_contract
-> construct_target=RESOURCE_CONTRACT
-> slot_target=output
-> executable=False
```

6.4 Validator 使用同一份 Role Contract

Validator 不应维护一份独立、不完整的 role contract。

Validator 应基于同一份 Role Contract 检查：

```
field 是否符合 semantic_role
route_family 是否符合 semantic_role
construct_target 是否符合 semantic_role
slot_target 是否符合 semantic_role
executable 是否符合 semantic_role
```

如果 Role Contract 明确要求某字段为 `None`，validator 也必须检查该字段确实为 `None`。

例如：

```
profile_domain:
    construct_target must be None
    slot_target must be None
```

不能把 `None` 当成“不检查”。

6.5 所有 annotation 生成路径共享同一契约

系统中可能有多种 annotation 来源：

- LLM refinement
deterministic packet annotation
adapter-derived annotation
legacy compatibility path

无论来源如何，最终 confirmed annotation 都必须符合统一 Role Contract。

这意味着：

- 不能一条路径使用 Role Contract；
另一条路径使用另一份 _ANNOTATION_SEMANTICS；
validator 又使用第三份 _ROLE_CONTRACT。

最终应形成：

- 单一 Role Contract source of truth
-> normalization
-> deterministic annotation derivation
-> validation
-> diagnostics

7. Role Contract 示例表

以下是架构层面的示例表。具体 role 集合可以随系统演进扩展，但每个 role 都必须有明确 contract。

semantic_role	field	route_family	construct_target	slot_target
profile_domain	domain	profile	None	None
input_contract	resources	resource_contract	RESOURCE_CONTRACT	input
output_contract	resources	resource_contract	RESOURCE_CONTRACT	output
process_step	behavior	flow_relevant	None	None
failure_mode	behavior	flow_relevant	EXCEPTION_FLOW	condition
failure_condition	behavior	flow_relevant	EXCEPTION_FLOW	condition
exception_handler_action	behavior	flow_relevant	EXCEPTION_FLOW	handler
worker_handoff_candidate	behavior	delegation_boundary	WORKER_HANDOFF	target
delegation_intent	behavior	delegation_boundary	WORKER_HANDOFF	target

semantic_role	field	route_family	construct_target	slot_target
handoff_condition	rules	delegation_boundary	WORKER_HANDOFF	condition
delegation_boundary_constraint	rules	delegation_boundary	CONSTRAINT	boundary
delegation_prohibition	rules	delegation_boundary	CONSTRAINT	prohibition
api_candidate	integrations	integration_candidate	API_CALL	target
integration_hint	integrations	integration_candidate	API_CALL	target
constraint	rules	constraint	None or CONSTRAINT	None

对于 contract 尚不稳定的 role，应采用保守策略：

可以作为 candidate / diagnostic hint;
不得直接 materialize construct;
不得让 LLM 自由指定 construct_target / slot_target。

8. Resource Contract Annotation 特殊边界

Resource contract annotation 是当前最敏感的场景，因为它会进入 DemandView、Stage 3.5、Stage 6 和 IRS。

8.1 Resource demand 的成立条件

Resource contract demand 只能来自：

semantic_role=input_contract
semantic_role=output_contract

不能来自：

construct_target=RESOURCE_CONTRACT
route_family=resource_contract
slot_target=input
slot_target=output

这些字段只能作为 consistency evidence。

8.2 禁止单字段入口

禁止以下逻辑作为 production 语义入口：

```
if construct_target == RESOURCE_CONTRACT:
    generate resource contract demand

if route_family == resource_contract:
    generate resource contract demand

if slot_target == input:
    generate input demand
```

8.3 DemandView 的接收规则

DemandView 只消费 validated resource contract annotations。

接收条件应包括：

```
semantic_role in {input_contract, output_contract}
role contract 派生字段一致
direction evidence 不冲突
requiredness 存在或显式 unspecified
provenance 可追踪
```

如果出现：

```
semantic_role=profile_domain
construct_target=RESOURCE_CONTRACT
slot_target=input
```

应产生 invalid annotation diagnostic，不得生成 demand。

9. Requiredness 的独立边界

`requiredness` 不属于 Role Contract 的简单派生字段。

原因是：

```
input_contract 可以是 required / optional / unspecified
output_contract 也可能需要表达 required / optional / unspecified
```

因此：

```
semantic_role=input_contract
```

只能派生：

```
direction=input
construct_target=RESOURCE_CONTRACT
slot_target=input
```

不能直接派生：

```
requiredness=required
```

9.1 requiredness 的合法来源

requiredness 应来自结构化证据，例如：

```
explicit required/optional schema
adapter hard facts
structured packet metadata
section kind that has been normalized into structured metadata
confirmed Stage 2 metadata
```

9.2 requiredness 的非法来源

禁止从以下来源推断 requiredness：

```
evidence_text.startswith("optional")
section title string matching
output_contract 默认 required
input_contract 默认 required
Stage 6 LLM 重新判断
IRS 根据 producer 状态反推
renderer 根据 REQUIRED/OPTIONAL 偏好补全
```

9.3 当前前置缺口

requiredness 三值要真正落地，必须先解决 upstream 数据源问题。

如果 canonical input、adapter evidence 或 Stage 2 annotation metadata 尚未提供 requiredness，那么下游即使支持三值，也只能看到：

```
requiredness=unspecified
```

或者继续依赖旧的 bool heuristic。

因此 requiredness 三值应作为独立演进轨道处理，不能与 Role Contract consolidation 混在同一个小变更里完成。

10. Validator 设计原则

Validator 是 Annotation 机制可靠性的关键边界。

Validator 应检查：

```
semantic_role 是否存在且合法
role contract 是否存在
field 是否符合 contract
route_family 是否符合 contract
construct_target 是否符合 contract
slot_target 是否符合 contract
executable 是否符合 contract
provenance 是否存在
requiredness 是否存在或显式 unspecified
multi-role / multi-direction 是否有结构化依据
```

Validator 不应：

```
调用 LLM
根据 raw text 修正语义
通过关键词补 role
静默把矛盾 annotation 改成另一种 role
静默丢弃矛盾 annotation 而不报告
```

对于可安全 normalize 的字段，validator 可以接受 role contract 覆盖，但必须保留观测信息。对于不可安全消解的矛盾，validator 应产生 diagnostic。

11. Diagnostic 策略

Annotation 错误必须可见。

典型 diagnostic kinds 包括：

```
annotation_role_contract_conflict
annotation_invalid_field_for_role
annotation_invalid_route_family_for_role
annotation_invalid_construct_target_for_role
annotation_invalid_slot_target_for_role
annotation_invalid_executable_for_role
annotation_missing_requiredness
annotation_coverage_gap
annotation_rejected_after_role_contract_validation
annotation_legacy_field_overridden_by_role_contract
```

诊断信息应至少表达：

```
span_id
semantic_role
raw conflicting field
expected field from role contract
source_section_id
source_packet_id
reason or source evidence
```

对于以下情况：

```
profile_domain + RESOURCE_CONTRACT/input
```

系统应产生：

```
annotation_role_contract_conflict
```

而不是：

```
静默生成 resource demand
静默改成 input_contract
静默丢弃且不报告
```

12. Prompt 的角色

Prompt 可以降低错误率，但不能作为 correctness guarantee。

Prompt 可以要求 LLM：

```
只判断 semantic_role;
不要决定 construct_target;
不要决定 slot_target;
不确定时输出 ambiguity;
不要为了匹配 Inputs/Outputs 关键词泛化其他 span。
```

但系统正确性必须由 deterministic Role Contract 和 Validator 保证。

设计边界是：

```
Prompt improves behavior.
Role Contract guarantees compiler consistency.
```

Validator exposes conflicts.
Downstream consumes only confirmed annotations.

13. 与 DemandView 的关系

DemandView 的职责保持不变：

从 resolved validated resource contract annotations
投影 source-demanded resource contract demands。

DemandView 不承担：

修复 Stage 2 annotation
根据 title/text 推断 demand
根据 construct_target 单字段生成 demand
根据 canonical facts 生成 demand
重新判断 requiredness

Role Contract 的价值是提高 DemandView 的输入质量。

最终关系应为：

Stage 2 LLM / deterministic source
-> semantic_role candidate

Role Contract Normalization
-> compiler-facing annotation fields

Annotation Validator
-> confirmed RouteAnnotation or diagnostic

Stage 3
-> resolved spans/routes

ResourceContractDemandView
-> source-demanded resource contract demands

DemandView 仍应保留自己的防御性规则。即使 Stage 2 已经 validator 过，DemandView 也不能接受：

construct_target=RESOURCE_CONTRACT

作为独立 demand 入口。

14. 与 CoverageValidator 的关系

Role Contract 和 CoverageValidator 解决不同问题。

Role Contract 解决：

已有 annotation 是否内部一致。

CoverageValidator 解决：

结构化源证据是否缺少 annotation。

CoverageValidator 可以读取 structural canonical facts 进行 coverage audit，但只能产生 diagnostics，不能生成 demand。

因此：

Role Contract prevents invalid annotation from becoming demand.
CoverageValidator prevents missing annotation from becoming silent loss.
DemandView prevents downstream single-field guessing.

三者共同防止：

静默误生成 demand
静默少生成 demand
静默消费矛盾 annotation

15. 多角色与多标签边界

部分 span 可能确实承担多个语义角色。例如：

一个异常处理动作也可能是普通 process step。
一个结构化 list item 可能同时提到 input 和 output。

但多角色不能由 LLM 单独声明成立。

合法多角色 / 多 demand 应依赖结构化依据，例如：

Stage 3 split 成不同 resolved spans
不同 source_packet_id

不同 list_item_id
Structural Adapter 明确标记 contains_multiple_contract_items

LLM 输出的:

explicit_multi_contract=true

不能单独作为授权依据。

16. 数据流

目标数据流如下:

```
Structured NL
-> Structural Adapter / Canonical input
-> Stage 1 spans
-> Stage 2 semantic_role classification
-> Role Contract normalization
-> Annotation validation
-> confirmed RouteAnnotation
-> Stage 3 resolved spans/routes
-> DemandView / ConstructPlan
-> Stage 3.5 worker boundary planning
-> Stage 6 materialization
-> IRS satisfaction checking
-> Renderer
```

核心变化是:

LLM 不再拥有 construct_target / slot_target / route_family 的最终写入权。

LLM 贡献语义分类, compiler 负责结构一致性。

17. 架构决策

最终架构决策如下:

1. Annotation 机制继续保留。
2. Role Contract 不是新机制, 而是已有机制的统一与强化。
3. semantic_role 是 LLM 的主要语义输出。
4. field、route_family、construct_target、slot_target、executable 由 Role Contract 确定性派生。
5. Role Contract 必须成为 normalizer、validator、deterministic annotation path 的统一 source of truth。

6. Validator 必须检查 full-field contract, 包括 expected `None`。
7. Downstream 不得通过 `construct_target` / `route_family` / `slot_target` 单字段推断 construct demand。
8. Resource contract demand 必须以 `input_contract` / `output_contract` semantic role 为主依据。
9. requiredness 不由 Role Contract 派生, 必须由结构化 metadata 或 explicit source 提供。
10. requiredness 数据源缺失时, 应显式保留 `unspecified` 或产生 diagnostic, 不得回退到文本启发式。
11. CoverageValidator 负责 missing annotation coverage, 不生成 demand。
12. DemandView 只消费 validated annotations。
13. Prompt 强化只是辅助, 不是 correctness guarantee。
14. 所有矛盾 annotation 必须可诊断。

18. 成功标准

本设计成功时, 系统应满足:

1. 不再出现 `profile_domain + RESOURCE_CONTRACT/input` 被下游误消费为 resource demand。
2. 不再存在多份互不一致的 role contract 表。
3. Validator 能检查 `construct_target=None` / `slot_target=None` 这类显式空值 expectation。
4. Deterministic annotation path 与 LLM refinement path 共享同一套 contract。
5. Resource contract demand 不再由 `construct_target=RESOURCE_CONTRACT` 单字段触发。
6. DemandView 只接收 validated `input_contract` / `output_contract` annotations。
7. `requiredness` 不再由 `_compute_required(direction, evidence_text)` 这类文本启发式决定。
8. 缺 `requiredness` 数据源时, 不静默投影为 required。
9. Coverage gap 通过 CoverageValidator 暴露, 而不是由 header fallback 补需求。
10. Prompt 修改不被视为唯一修复。
11. Annotation diagnostic 能进入最终报告或中间诊断链路。
12. 下游阶段消费的是 compiler-confirmed annotation, 而不是 raw LLM object。

19. 核心原则

最终系统必须遵守:

```
LLM decides semantic_role.  
Role Contract derives compiler fields.  
One Role Contract source of truth.  
Validator checks full-field consistency.  
Expected None is still a contract.  
Requiredness is independent tri-state semantics.  
DemandView consumes only validated resource contract annotations.  
CoverageValidator detects missing annotations but does not generate demands.  
No construct_target single-field inference.  
No route_family single-field inference.  
No slot_target single-field direction inference.  
No hidden fallback.  
No downstream guessing.  
No silent invalid annotation consumption.
```